

ETIN35—Manual for RISC-V project

Love Bárány
love.barany@eit.lth.se

March 2026

Contents

1	Introduction, description of project, and some general tips	1
2	More detailed requirements per grade level	2
2.1	Grade 3	2
2.2	Grade 4	3
2.3	Grade 5	3
3	Useful material to consider	3
3.1	Basic 5-stage pipeline implementations	3
3.2	Computer Organization and Design RISC-V Edition, The Hardware Software Interface	4
3.3	Computer Architecture — A Quantitative Approach	4
3.4	The RISC-V Reader: An Open Architecture Atlas	5
3.5	The RISC-V Instruction Set Manual Volume I: Unprivileged ISA	5
4	Instruction chart with grade levels marked	5

1 Introduction, description of project, and some general tips

This document provides a more detailed manual regarding the second-phase project *Build your own RISC-V*, for the course **ETIN35** IC-project 1 at LTH, Lund University. As the name suggests, the goal of the project is to build or create your own RISC-V processor that implements a subset of instructions and various performance enhancements. Exactly which instructions, and which enhancements, are listed in the next section.

Your pipeline should consist of 5 stages, be 32-bit, and have a control unit. It should be able to receive new programs over a serial link from a computer, to the FPGA the processor is implemented on. This is done through the UART protocol, which can be used with a USB cable. You can find more information about the UART protocol online, for instance [here](#).

2 More detailed requirements per grade level

This section is accompanied by an annotated RISC-V reference card, where the instructions to implement are marked with a colour per grade level. This chart can be found at the end of this manual, but is also attached as a separate document. For all grade levels, the functionalities listed should be implemented and verified on an FPGA. By implemented, it means that you should be able to synthesize, implement/PnR, and generate a bitstream for the design without issue, and then transfer the bitstream to the FPGA and run it there. By verify, it means that you should be able to run various programs on the FPGA and be able to see that they have worked as expected. The programs should be transferred from a computer via a USB cable, using the UART protocol. The recommended way to verify if your programs are executing correctly is to use the Integrated Logic Analyzer IP Vivado comes with by default, **see Xilinx documentation here**. I recommend you place ILAs in your register file, program memory, and data memory, but you can also place them anywhere you want to easily debug.

There are certain exceptions that can occur based on the instructions you choose to implement, for instance: division by 0, or an incorrect instruction. Since your processor will not have any operating system to throw the exception at, you don't need to handle them in any particular way other than freezing the pipeline for example. Your processor should give some indication that an exception has occurred, for instance by turning on a specific LED on the board. It is up to you to decide exactly how you want to do it, an ILA is also fine for instance.

During approvals, I will run various programs that test the instructions and control units, and I will also go through your code and discuss it with you.

At the end of the term, all groups in each project should hold a presentation together, describing what has been done. It is also interesting to bring up differences between implementations here. Finally, a written report should be written where you describe what you've done in the project, where you also discuss why you chose to implement things in certain ways. More information about the presentation and report will come later, but it's a good idea to document your decisions as you go!

2.1 Grade 3

- Implement the instructions marked by green in the accompanying reference card.
- Implement stalling, hazard detection, and forwarding. If a hazard is detected which cannot be forwarded, the pipeline should be stalled.
- You should integrate the serial UART controller into your design. There is a finished controller that is provided to you (see later section), but it is only a controller: it doesn't write programs to your program memory.

- Implement a simple 2-bit saturation counter branch predictor. It should only be based on local history. You can decide the size of the branch history table/branch prediction buffer yourselves.

2.2 Grade 4

- Implement the instructions marked by orange in the accompanying reference card.
- The multiplication and division instructions should not be done in one cycle. Instead, you should implement some way of handling multi-cycle instructions. An easy approach is to stall the pipeline while you wait for the result.
- You should implement a more advanced branch predictor scheme, a 2-level/correlating predictor. There are many, I recommend **gshare**.

2.3 Grade 5

- Implement the instructions marked by red in the accompanying reference card.
- Floating-point numbers need to be stored in a different way. Extend your pipeline to handle this, with a second set of registers for floating-point values. As with multiplication and division, your instructions will most likely need to take more than one cycle.
- You should implement an adaptive branch predictor. I recommend the Tournament predictor described in the course book.

3 Useful material to consider

There is a lot of information available by a quick Google search, of varying quality. There is a lot of good things that can be found, but to facilitate your searching slightly I recommend the following material. You should be able to complete the project solely based on these, but if you need further material there is nothing stopping you from seeking it out on your own!

3.1 Basic 5-stage pipeline implementations

A previous PhD student created a basic 5-stage pipeline RISC-V, which you can start out from! It provides the basic building blocks in the forms of various modules which should be expanded, and integrated with new ones. You can start from absolutely zero if you would rather do that, but these can still serve as good inspiration.

The basic RISC-V machines are available on GitHub, implemented in SystemVerilog and VHDL:

- github.com/masoud-ata/riscv_sv (SystemVerilog)

Something to note is that as previously mentioned you should be able to send programs to your RISC-V via UART over USB.

This processor already has some structure set up, and some instructions are implemented (see the ones marked blue in the accompanying reference card).

3.2 Computer Organization and Design RISC-V Edition, The Hardware Software Interface

This book, written by David A. Patterson and John L. Hennessy, provides implementation details of a RISC-V pipeline, and is extremely useful for this project. The first few chapters introduce some computer architecture concepts, assembly language and instructions, and some arithmetic for computers. The main chapter is Chapter 4 titled *The Processor*, which describes building a pipeline from the ground. The two basic implementations in Subsection 3.1 are based on this chapter. If you choose to start from zero, it is highly recommended that you follow this chapter. Even if you choose to start from the base implementations, you should read through this chapter to understand how the pipelines are implemented.

The remaining chapters are also interesting, but not as relevant to this project. Appendix A goes through basic digital logic design, and Appendix C describes how to go from control description on paper to digital implementations (this could be seen as a quick refresher of the first VLSI course).

This book is available for free through your LU-account. Go to finn.lub.lu.se, log in with your LU-account, and search for the book.

3.3 Computer Architecture — A Quantitative Approach

This is the book that was used in the Computer Architecture course. It's much more focused on theory than the previous, but that doesn't mean it isn't useful! It is also written by the same authors, Hennessy and Patterson. I would recommend looking at this one when implementing the various branch predictors, since it is more theoretical in nature, and Computer Organization and Design doesn't go into as much detail regarding this. Section C.2 (in the 6th edition) talks of hazards in general, and describes a more simple dynamic branch predictor. Section 3.3 (in the 6th edition) talks about more advanced branch predictors. Section C.3 (in the 6th edition) covers some basic pipeline implementation details, which Computer Organization and Design also covers.

This book is also available for free through your LU-account. The 5th edition is available as an e-book, and various physical libraries around LTH also carry copies of the book. Go to finn.lub.lu.se and search for it after logging in with your LU-account.

3.4 The RISC-V Reader: An Open Architecture Atlas

This book is also written by Patterson, but Hennessy was not involved with this one (are you starting to recognize some names?). This book is great for practical details. It contains more detailed information about all of the instructions that you will implement. Its chapters are divided into the different ISAs, so depending on what grade you're going for you'll need more or less of this book. There are of course other sources for what the different instructions do, however this one is written well in my opinion. Appendix A lists all instructions of RV{I,C,M,F} and more, is easy to search and also explains through more software-like code what each instruction does. The website for this book is www.riscvbook.com where you can get free PDFs of the book in Chinese, Spanish, Portuguese, or Korean, but you could also just make a quick google search of the form “`the risc-v reader pdf`” and see what you find! The book is also available on Amazon if you so desire.

3.5 The RISC-V Instruction Set Manual Volume I: Unprivileged ISA

Probably the most technical “book” of the bunch. It is similar to the RISC-V Reader in that it lists all instructions, but this document lists *all* the instructions. This is not a document you scroll through leisurely, it is recommended to know what you're looking for and use the search function to find more. You can find its glorious 727 pages of technical details at [at this URL](https://riscv.org/specifications/unprivileged-isa/).

4 Instruction chart with grade levels marked

